

Module 2 FEM Implementation Note

Detailed How-To Guide for RadiationEffects_proj2

Running, testing, inspecting, modifying, and diagnosing the two-dimensional electrostatic finite-element solver

Purpose and scope. This is the operational companion to the Module 2 physics note. It documents the MATLAB finite-element method (FEM) implementation of the two-dimensional electrostatic Poisson problem: what to run, which files are called, which parameters control the calculation, what each returned field means, how to execute the verification tests, and how to probe the numerical system when a result is unexpected. It documents only the FEM path beginning at `main_module2_electrostatics.m`. The separate Module 2 physics-informed neural-network (PINN) implementation will receive its own implementation note.

Fastest correct start. Open MATLAB in the repository's `matlab/` directory, run `setup_project_paths`, and then run `result = main_module2_electrostatics('localized_defect_charge');`. The default driver saves a MAT-file, three PNG figures, and a text summary in `matlab/outputs/module2_2d/`.

Contents

Symbols and acronyms used in this document	3
1 What the Module 2 FEM code does	4
1.1 Implemented capabilities	5
1.2 Important current limitations	5
2 Software requirements and starting location	6
2.1 Required software	6
2.2 Correct working directory	6
2.3 Path preflight check	7
3 Quick-start execution recipes	7
3.1 Default run	7

3.2	Run all four built-in FEM cases	7
3.3	Run one case from a system terminal	7
3.4	Run without writing plots or a MAT-file	8
4	Built-in cases and expected behavior	8
5	File map and responsibility of each component	9
6	Exact execution sequence	11
7	Parameter reference	11
7.1	Physical constants and material	11
7.2	Geometry and mesh	12
7.3	Carrier, dopant, and defect charge	12
7.4	Boundary conditions	14
7.5	Output controls	14
8	Custom runs without editing the baseline defaults	15
8.1	Mesh-refined localized source	15
8.2	Uniform dopant-only source	15
8.3	Saving a direct-solver result deliberately	15
9	Returned data structure: what to inspect	16
10	How to probe a run numerically	16
10.1	Dimensions, ranges, and finiteness	16
10.2	Linear-system residual	17
10.3	Boundary-condition error	17
10.4	Matrix structure and conditioning	17
10.5	Probe the node nearest a physical coordinate	18
10.6	Inspect one triangle	18
11	Generated outputs and how to check them	18

12 Verification tests	19
12.1 Run the FEM-only test suite	19
12.2 What each test proves	20
12.3 Batch-test command	20
12.4 Mesh-refinement check	21
13 Debugging and interactive code tracing	21
13.1 Stop at the first MATLAB error	21
13.2 Enter the solver at its first line	21
13.3 Inspect source code and resolution	22
14 Common failure modes and diagnostic actions	22
15 Adding a new standalone case safely	23
15.1 One-off experiment	24
15.2 New named regression or demonstration case	24
16 Preparing for Module 3 coupling	24
17 Recommended verification ladder before using a result	25
18 Minimal handoff checklist	26
19 Next implementation note	26

Symbols and acronyms used in this document

Symbol / acronym	Name	Meaning in this document
FEM	Finite-element method	Galerkin discretization using three-node linear triangular elements.
T3	Three-node triangle	Linear triangular element with one potential degree of freedom at each vertex.

Symbol / acronym	Name	Meaning in this document
Ω	Computational domain	Rectangle $[0, L_x] \times [0, L_y]$.
ϕ	Electrostatic potential	Primary FEM unknown, stored at mesh nodes in volts.
$\mathbf{E}$	Electric field	Postprocessed as $\mathbf{E} = -\nabla\phi$, in volts per meter.
ρ	Space-charge density	Volumetric source in coulombs per cubic meter.
ε_{Si}	Silicon permittivity	Product of relative and vacuum permittivity, in farads per meter.
N_a	Shape function	Linear basis function for local triangle node a .
K	Stiffness matrix	Unconstrained sparse global Poisson matrix.
K_{bc}	Boundary-modified matrix	Matrix after strong Dirichlet constraints are imposed.
$\mathbf{b}$	Source vector	Assembled right-hand side before boundary modification.
Φ	Nodal potential vector	Solution of $K_{\text{bc}}\Phi = \mathbf{b}_{\text{bc}}$.
BC	Boundary condition	Dirichlet potential or natural homogeneous Neumann field condition.
SI	International System of Units	Unit convention used by all Module 2 parameters.

1 What the Module 2 FEM code does

The solver computes the electrostatic potential generated by mobile carriers, ionized dopants, and an effective charged-defect population. The strong-form equation is

$$\nabla \cdot (\varepsilon_{\text{Si}} \nabla \phi) = -\rho, \quad \mathbf{E} = -\nabla \phi. \quad (1.1)$$

The implemented weak form is

$$\int_{\Omega} \varepsilon_{\text{Si}} \nabla w \cdot \nabla \phi \, d\Omega = \int_{\Omega} w \rho \, d\Omega, \quad (1.2)$$

with prescribed potential on selected boundaries and zero normal potential gradient on natural boundaries. The FEM approximation produces

$$K\Phi = \mathbf{b}. \quad (1.3)$$

The current path is a **steady, linear, single-material electrostatic solve**. There is no time step in Module 2 itself. Time dependence enters a future coupled calculation only because Module 3, Module 4, and Module 5 can supply a new charge state at successive coupled times.

1.1 Implemented capabilities

- Structured rectangular node placement with two counter-clockwise triangles per rectangular cell.
- Constant silicon permittivity throughout the domain.
- Nodal charge density from electrons, holes, ionized dopants, and one effective Gaussian defect population.
- A special uniform-charge source used for analytical verification.
- Strong, constant-valued Dirichlet conditions on any named rectangular side.
- Natural homogeneous Neumann conditions on sides that are not Dirichlet.
- Sparse global assembly, direct sparse solution with MATLAB's backslash operator, element electric fields, and area-weighted nodal electric fields.
- MAT-file, PNG-figure, and text-summary output through the top-level driver.

1.2 Important current limitations

- The implementation does *not* assemble a nonzero Neumann boundary integral. The fields `params.bc.*.dphidn` are descriptive placeholders for the present zero-flux cases. Changing one to a nonzero value does not change the solution.
- Permittivity is supplied as one scalar. Spatially varying or tensor permittivity is not yet supported.
- Dirichlet values are scalar constants per side; a spatially varying contact profile is not accepted by the current boundary collector.
- The default defect source is generated internally. A nodal defect map from Module 3 is not yet an input option in this standalone source builder.
- The solver does not solve carrier statistics or trap occupancy self-consistently. It accepts concentrations and an effective defect charge number as prescribed inputs.

- An all-Neumann problem is singular. At least one potential reference constraint is required; the standard cases constrain both left and right sides.

2 Software requirements and starting location

2.1 Required software

The Module 2 FEM path calls core MATLAB operations only: structures, sparse matrices, the backslash linear solver, basic plotting, file I/O, and path management. It does not call the Deep Learning Toolbox and does not require the Module 2 PINN files. Geant4 is also not required for the four standalone FEM cases.

The practical requirements are:

- MATLAB with permission to read the repository and write under `matlab/outputs/`;
- sufficient memory for the requested mesh;
- a graphics-capable MATLAB installation when `makePlots=true`. The figures are created with `'Visible','off'`, so an interactive figure window is not required in the usual batch configuration.

2.2 Correct working directory

The least ambiguous starting point is the project's MATLAB directory:

```
1 cd('C:/path/to/RadiationEffects_proj2/matlab')
2 setup_project_paths
```

Replace the example path with the actual extracted repository path. Forward slashes work in MATLAB on Windows. If MATLAB was opened directly in `matlab/`, only the second line is needed.

`setup_project_paths.m` adds the following directories:

- `matlab/`;
- `matlab/cases/`;
- `matlab/tests/`;
- `matlab/pinn/`;
- all folders recursively below `matlab/src/`.

The PINN directory is added for the complete project, but it is not used by the FEM run documented here.

2.3 Path preflight check

Before solving, confirm that MATLAB resolves the intended files:

```
1 which main_module2_electrostatics -all
2 which solve_poisson_defect_space_charge_2d -all
3 which default_module2_params -all
```

Each command should identify the corresponding file inside the current project's `matlab/` tree. Multiple results can indicate that an older project copy is still on the MATLAB path. Remove the stale path or start a fresh MATLAB session before comparing results.

3 Quick-start execution recipes

3.1 Default run

Calling the driver without an argument selects the localized charged-defect case:

```
1 setup_project_paths
2 result = main_module2_electrostatics;
```

This is equivalent to

```
1 result = main_module2_electrostatics('localized_defect_charge');
```

3.2 Run all four built-in FEM cases

```
1 setup_project_paths
2
3 r0 = main_module2_electrostatics('zero_charge');
4 r1 = main_module2_electrostatics('linear_potential');
5 r2 = main_module2_electrostatics('uniform_space_charge');
6 r3 = main_module2_electrostatics('localized_defect_charge');
```

The same output filenames are overwritten when the same case is rerun. Distinct built-in case names do not overwrite one another because the case name prefixes every generated file.

3.3 Run one case from a system terminal

MATLAB's batch interface can run the solver without first opening the desktop:

```
matlab -batch "cd('C:/path/to/RadiationEffects_proj2/matlab'); setup_project_paths;
result = main_module2_electrostatics('localized_defect_charge');"
```

The terminal process returns a nonzero exit status if MATLAB encounters an uncaught error, which is useful for automated verification.

3.4 Run without writing plots or a MAT-file

The top-level driver constructs its own default parameter structure, so output flags cannot be passed through its function signature. For a diagnostic solve with reduced file output, call the parameter factory and solver directly:

```
1 setup_project_paths
2 params = default_module2_params('localized_defect_charge');
3 params.makePlots = false;
4 params.saveMat = false;
5 result = solve_poisson_defect_space_charge_2d(params);
```

In this direct-solver path, `makePlots` and `saveMat` are not read because plotting and saving belong to the top-level driver. The flags are still useful when the same `params` structure is later passed through a custom driver. The direct call itself writes no files unless plotting and summary functions are invoked explicitly.

4 Built-in cases and expected behavior

Case name	Configuration	Expected result and purpose
<code>zero_charge</code>	$\rho = 0$; left and right potentials both zero.	$\phi = 0$ and $\mathbf{E} = 0$ to numerical precision. This detects unintended sources or boundary values.
<code>linear_potential</code>	$\rho = 0$; left potential 0 V and right potential 1 V.	Exact Laplace solution $\phi = x/L_x$ with uniform $E_x = -1/L_x$. This checks mesh assembly, Dirichlet insertion, and field sign.
<code>uniform_space_charge</code>	Explicit $\rho_0 = 5.0 \times 10^{-4}$ C/m ³ ; left and right potentials zero.	Nearly one-dimensional quadratic solution $\phi = \rho_0 x(L_x - x)/(2\epsilon_{Si})$. This checks the source vector.

Case name	Configuration	Expected result and purpose
localized_defect_charge	Positive Gaussian effective defect population centered in the device; grounded left and right contacts.	Finite positive potential perturbation and localized positive charge. This is a physical sanity case, not an exact-solution verification.

The files in `matlab/cases/` are small convenience functions that return these parameter structures. The main driver does not call the case functions; it calls `default_module2_params(caseName)` directly. For example,

```
1 params = case_module2_linear_potential_2d();
2 result = solve_poisson_defect_space_charge_2d(params);
```

is a valid direct-solver route, but it bypasses the driver's automatic saving, plotting, summary writing, and console report.

5 File map and responsibility of each component

File	Responsibility
<code>matlab/main_module2_electrostatics.m</code>	User-facing driver. Selects a built-in case, runs the solver, creates the output directory, saves results, makes plots, writes a summary, and prints scalar diagnostics.
<code>matlab/setup_project_paths.m</code>	Adds the driver, cases, tests, PINN directory, and all source subdirectories to the MATLAB path.
<code>matlab/src/module2_electrostatics/default_module2_params.m</code>	Defines constants, geometry, mesh density, source parameters, boundary conditions, output controls, and the four supported case modifications.
<code>matlab/src/module2_electrostatics/solve_poisson_defect_space_charge_2d.m</code>	Numerical orchestrator. Calls mesh generation, charge evaluation, assembly, boundary processing, sparse solve, and field postprocessing.
<code>matlab/src/module2_electrostatics/make_rectangular_tri_mesh_2d.m</code>	Creates nodes, counter-clockwise T3 connectivity, coordinate vectors, dimensions, and left/right/top/bottom boundary-node sets.

File	Responsibility
matlab/src/module2_electrostatics/build_space_charge_module2_2d.m	Evaluates the nodal source ρ . Uses explicit <code>rho_uniform</code> when present; otherwise evaluates carriers, dopants, and a Gaussian effective defect population.
matlab/src/module2_electrostatics/assemble_poisson_fem_2d.m	Loops over triangles, forms local stiffness and source contributions, scatters them into a sparse global matrix and right-hand side, and stores element areas and centroids.
matlab/src/module2_electrostatics/compute_element_stiffness_triangle.m	Calculates signed area, shape-function gradients, and $K_e = \varepsilon A_e (\nabla N)(\nabla N)^T$. Rejects non-positive element orientation.
matlab/src/module2_electrostatics/compute_element_source_triangle.m	Calculates the consistent T3 source vector using the triangle mass-like matrix multiplied by the three nodal charge values.
matlab/src/module2_electrostatics/get_module2_dirichlet_nodes.m	Collects nodes and scalar values from sides whose type is 'dirichlet'. Resolves duplicate corner nodes in side order: left, right, bottom, top.
matlab/src/module2_electrostatics/apply_dirichlet_bc.m	Adjusts the right-hand side for known values, zeros constrained rows and columns, and inserts unit diagonal equations.
matlab/src/module2_electrostatics/compute_electric_field_from_potential_2d.m	Computes elementwise $-\nabla\phi$ and area-weighted nodal averages.
matlab/src/module2_electrostatics/plot_module2_result_2d.m	Writes potential, space-charge, and electric-field-magnitude PNG figures.
matlab/src/module2_electrostatics/write_module2_summary.m	Writes geometry, mesh, material, source, potential, field, and boundary-node scalar values to a text file.
matlab/cases/case_module2_*.m	Convenience parameter factories for the four built-in cases.
matlab/tests/test_module2_*.m	Standalone FEM verification and sanity tests. PINN and CPINN tests with similar prefixes are outside this note's scope.

6 Exact execution sequence

For `main_module2_electrostatics('localized_defect_charge')`, the call sequence is:

1. The driver checks the input and supplies 'localized_defect_charge' when no name is given.
2. `default_module2_params` creates the complete parameter structure and applies the selected case branch.
3. `solve_poisson_defect_space_charge_2d` creates the mesh.
4. `build_space_charge_module2_2d` evaluates one charge value per node.
5. `assemble_poisson_fem_2d` forms unconstrained `K` and `rhs`.
6. `get_module2_dirichlet_nodes` resolves the constrained node list and potential values.
7. `apply_dirichlet_bc` forms `Kbc` and `rhsbc`.
8. MATLAB solves $\text{phi} = \text{Kbc} \setminus \text{rhsbc}$.
9. `compute_electric_field_from_potential_2d` computes element and nodal fields.
10. The solver returns a `result` structure containing inputs, matrices, solution, fields, and diagnostics.
11. The driver saves the MAT-file, creates three figures, writes the text summary, and prints a completion report.

This separation is useful while debugging: call the solver directly to isolate numerical behavior, and call the driver when testing the complete input/output workflow.

7 Parameter reference

All physical inputs are in SI units unless noted otherwise. The default values below are defined before the case-specific switch is evaluated.

7.1 Physical constants and material

Field	Default	Units	Meaning
q	1.602176634 10^{-19}	$\times$ C	Elementary charge magnitude.

Field	Default	Units	Meaning
eps0	$8.8541878128 \times 10^{-12}$	F/m	Vacuum permittivity.
eps_rel_si	11.7	dimensionless	Relative permittivity of silicon.
eps_si	eps_rel_si*eps0	F/m	Scalar permittivity passed to element assembly. Recompute it after changing either factor.

7.2 Geometry and mesh

Field	Default	Units	Meaning
Lx	10×10^{-6}	m	Domain length in x .
Ly	4×10^{-6}	m	Domain length in y .
nx	41	nodes	Number of structured node locations along x . Must be at least two.
ny	21	nodes	Number of structured node locations along y . Must be at least two.

The node count is $N = n_x n_y$, and the triangle count is

$$N_e = 2(n_x - 1)(n_y - 1). \quad (7.1)$$

Increasing **nx** and **ny** improves spatial resolution but increases matrix size, assembly time, solution time, and output size.

7.3 Carrier, dopant, and defect charge

Field	Default	Units	Meaning
n	0	m ⁻³	Uniform electron concentration, entering with a negative sign.
p	0	m ⁻³	Uniform hole concentration, entering with a positive sign.
ND_plus	0	m ⁻³	Uniform ionized donor concentration.
NA_minus	0	m ⁻³	Uniform ionized acceptor concentration.
Cdef_background	0	m ⁻³	Uniform effective defect concentration.
Cdef_peak	2.0×10^{20}	m ⁻³	Peak Gaussian defect concentration before the case switch.
Cdef_x0	$0.5L_x$	m	Gaussian center in x .
Cdef_y0	$0.5L_y$	m	Gaussian center in y .
Cdef_sigma_x	$0.10L_x$	m	Gaussian standard deviation in x . Must be positive.
Cdef_sigma_y	$0.15L_y$	m	Gaussian standard deviation in y . Must be positive.
zdef	1.0	dimensionless	Effective charge number per defect. Positive gives positive defect charge.
rho_uniform	case-specific	C/m ³	When this field exists, it bypasses all carrier, dopant, and defect calculations and supplies an explicit uniform source.

Unless **rho_uniform** exists, the source builder evaluates

$$C_{\text{def}}(x, y) = C_{\text{bg}} + C_{\text{peak}} \exp \left[-\frac{1}{2} \left(\frac{x - x_0}{\sigma_x} \right)^2 - \frac{1}{2} \left(\frac{y - y_0}{\sigma_y} \right)^2 \right], \quad (7.2)$$

$$\rho(x, y) = q \left[p - n + N_D^+ - N_A^- + z_{\text{def}} C_{\text{def}}(x, y) \right]. \quad (7.3)$$

Common source pitfall. If a parameter structure was created for `uniform_space_charge`, deleting or changing carrier/defect fields will have no effect while `rho_uniform` still exists. Remove it with `params = rmfield(params,'rho_uniform')` before using the general source expression.

7.4 Boundary conditions

Each side has a `type`. A Dirichlet side also has `value`; a nominal Neumann side has `dphidn`. The defaults are grounded left and right contacts with natural top and bottom boundaries:

```
1 params.bc.left.type = 'dirichlet';
2 params.bc.left.value = 0.0;
3 params.bc.right.type = 'dirichlet';
4 params.bc.right.value = 0.0;
5 params.bc.bottom.type = 'neumann';
6 params.bc.bottom.dphidn = 0.0;
7 params.bc.top.type = 'neumann';
8 params.bc.top.dphidn = 0.0;
```

Only the `'dirichlet'` branches are explicitly processed. All other sides contribute the natural homogeneous boundary term of the weak form. A nonzero `dphidn` value is not yet integrated.

Corner nodes can belong to two Dirichlet sides. The collector visits sides in the order left, right, bottom, top and retains the first value encountered for a duplicated node. Avoid incompatible values at a shared corner unless this precedence is intentional.

7.5 Output controls

Field	Default	Effect in the top-level driver
<code>outputDir</code>	<code>outputs/ module2_2d</code>	Resolved relative to the directory containing the top-level driver.
<code>makePlots</code>	<code>true</code>	Calls <code>plot_module2_result_2d</code> .
<code>saveMat</code>	<code>true</code>	Saves the complete <code>result</code> structure.

The text summary is always written by the top-level driver, independent of `makePlots` and `saveMat`.

8 Custom runs without editing the baseline defaults

For exploratory work, create a parameter structure, override fields in the command window or a new script, and call the numerical solver directly. This preserves the validated defaults.

8.1 Mesh-refined localized source

```

1  setup_project_paths
2  params = default_module2_params('localized_defect_charge');
3  params.nx = 81;
4  params.ny = 41;
5  params.Cdef_peak = 5.0e20;
6  params.zdef = -1.0;
7  result = solve_poisson_defect_space_charge_2d(params);
8
9  fprintf('Potential range: [%g, %g] V\n', ...
10         min(result.phi), max(result.phi));

```

Changing `zdef` from `+1` to `-1` reverses the defect-charge contribution. With symmetric grounded contacts and no other charge, the potential perturbation should reverse sign.

8.2 Uniform dopant-only source

```

1  params = default_module2_params('zero_charge');
2  params.ND_plus = 1.0e20;
3  params.NA_minus = 0.0;
4  result = solve_poisson_defect_space_charge_2d(params);

```

Because the concentrations are treated as prescribed uniform values, this is a source-driven electrostatic experiment, not a self-consistent semiconductor equilibrium calculation.

8.3 Saving a direct-solver result deliberately

```

1  outputDir = fullfile(pwd, 'outputs', 'module2_2d', 'custom_probe');
2  if ~exist(outputDir, 'dir')
3      mkdir(outputDir);
4  end
5  save(fullfile(outputDir, 'custom_probe_results.mat'), 'result');
6  plot_module2_result_2d(result, outputDir);
7  write_module2_summary(result, outputDir);

```

Assign a unique `params.caseName` before plotting or summary writing if several custom cases share the same output directory.

9 Returned data structure: what to inspect

The numerical solver returns a structure with the following top-level fields.

Field	Contents
<code>result.params</code>	Complete parameter structure used for the solve.
<code>result.mesh</code>	Nodes, element connectivity, coordinate vectors, dimensions, domain lengths, and four boundary-node sets.
<code>result.rho</code>	$N \times 1$ nodal charge-density vector.
<code>result.K</code>	Unconstrained sparse stiffness matrix.
<code>result.rhs</code>	Unconstrained assembled source vector.
<code>result.Kbc</code>	Sparse matrix after Dirichlet insertion.
<code>result.rhsbc</code>	Boundary-modified right-hand side.
<code>result.fixedNodes</code>	Constrained global node indices.
<code>result.fixedValues</code>	Prescribed potentials corresponding to <code>fixedNodes</code> .
<code>result.phi</code>	Solved nodal potential vector.
<code>result.field</code>	Element and nodal electric-field arrays, magnitudes, centroids, and areas.
<code>result.elementData</code>	Element areas and centroids recorded during assembly.
<code>result.maxAbsPhi</code>	Maximum absolute nodal potential.
<code>result.maxAbsE</code>	Maximum area-averaged nodal field magnitude.

Useful mesh subfields include `nodes` ($N \times 2$ coordinates), `elems` ($N_e \times 3$ connectivity), `x`, `y`, `nx`, `ny`, and `boundary.left/right/top/bottom`. Useful field subfields include `Eelem`, `Ex_elem`, `Ey_elem`, `Enodal`, `Ex_nodal`, `Ey_nodal`, `Emag_elem`, and `Emag_nodal`.

10 How to probe a run numerically

10.1 Dimensions, ranges, and finiteness

```
1 size(result.mesh.nodes)
2 size(result.mesh.elems)
```



```

3 size(result.Kbc)
4
5 [min(result.rho), max(result.rho)]
6 [min(result.phi), max(result.phi)]
7 [min(result.field.Ex_nodal), max(result.field.Ex_nodal)]
8 [min(result.field.Ey_nodal), max(result.field.Ey_nodal)]
9
10 assert(all(isfinite(result.rho)))
11 assert(all(isfinite(result.phi)))
12 assert(all(isfinite(result.field.Enodal), 'all'))

```

For MATLAB releases that do not accept 'all' in this reduction, replace the final assertion with the equivalent vectorized form:

```

1 assert(all(isfinite(result.field.Enodal(:))))

```

10.2 Linear-system residual

After boundary insertion, the computed solution should satisfy the modified system to floating-point accuracy:

```

1 algebraicResidual = result.Kbc * result.phi - result.rhsbc;
2 relativeResidual = norm(algebraicResidual, 2) / ...
3     max(norm(result.rhsbc, 2), eps);
4 fprintf('Relative algebraic residual = %.3e\n', relativeResidual);

```

This checks whether the linear system was solved. It does *not* by itself prove that the PDE, source, or boundary conditions were assembled correctly.

10.3 Boundary-condition error

```

1 bcError = max(abs(result.phi(result.fixedNodes) - ...
2     result.fixedValues));
3 fprintf('Maximum Dirichlet error = %.3e V\n', bcError);

```

10.4 Matrix structure and conditioning

```

1 figure; spy(result.Kbc); title('Boundary-modified FEM sparsity');
2 symmetryDefect = norm(result.Kbc - result.Kbc.', 'fro') / ...
3     max(norm(result.Kbc, 'fro'), eps);

```

```
4 conditionEstimate = condest(result.Kbc);
```

The unconstrained electrostatic stiffness matrix is symmetric. The boundary routine zeros both constrained rows and columns, so Kbc should also be symmetric to roundoff. A very large condition estimate can accompany poor scaling, extreme geometry aspect ratio, or excessive mesh refinement.

10.5 Probe the node nearest a physical coordinate

```
1 target = [0.50*result.params.Lx, 0.50*result.params.Ly];
2 distance2 = sum((result.mesh.nodes - target).^2, 2);
3 [~, nodeID] = min(distance2);
4
5 fprintf('Node %d at (%g,%g) m\n', nodeID, ...
6       result.mesh.nodes(nodeID,1), result.mesh.nodes(nodeID,2));
7 fprintf('rho = %g C/m^3, phi = %g V\n', ...
8       result.rho(nodeID), result.phi(nodeID));
9 fprintf('E = (%g,%g) V/m\n', ...
10      result.field.Ex_nodal(nodeID), ...
11      result.field.Ey_nodal(nodeID));
```

10.6 Inspect one triangle

```
1 elementID = 1;
2 localNodes = result.mesh.elems(elementID,:);
3 coordinates = result.mesh.nodes(localNodes,:);
4 [Ke, area, gradN] = compute_element_stiffness_triangle( ...
5     coordinates, result.params.eps_si);
6 localRho = result.rho(localNodes);
7 fe = compute_element_source_triangle(area, localRho);
```

This is the most direct way to study the connection between triangle geometry, basis gradients, element matrix, source values, and global assembly.

11 Generated outputs and how to check them

For a case named <case>, the default driver writes:

- <case>_results.mat: complete result structure;
- <case>_potential.png: nodal potential on the triangular mesh;

- `<case>_space_charge.png`: nodal space-charge density;
- `<case>_electric_field_magnitude.png`: area-averaged nodal field magnitude;
- `<case>_summary.txt`: scalar run summary.

The output directory is `matlab/outputs/module2_2d/`. After a run, check:

1. the summary reports the requested domain and mesh;
2. the number of nodes is `nx*ny`;
3. the number of triangles is `2*(nx-1)*(ny-1)`;
4. charge, potential, and field ranges are finite;
5. the potential agrees with the prescribed left and right values;
6. plot symmetry or sign agrees with the selected source;
7. the MAT-file loads and reproduces the scalar values in the summary.

```
1 loaded = load(fullfile('outputs', 'module2_2d', ...  
2     'localized_defect_charge_results.mat'));  
3 loaded.result.maxAbsE
```

12 Verification tests

12.1 Run the FEM-only test suite

From `matlab/`:

```
1 setup_project_paths  
2 test_module2_zero_charge_2d  
3 test_module2_linear_potential_2d  
4 test_module2_uniform_space_charge_2d  
5 test_module2_localized_defect_charge_2d
```

The following similarly named tests validate different code paths and are outside this FEM test group:

- `test_module2_pinn_electrostatics`;
- `test_module2_cpinn_electrostatics`.

Those tests can require the Deep Learning Toolbox.

To stop on the first failure while running the FEM group programmatically:

```
1 femTests = { ...
2     @test_module2_zero_charge_2d, ...
3     @test_module2_linear_potential_2d, ...
4     @test_module2_uniform_space_charge_2d, ...
5     @test_module2_localized_defect_charge_2d};
6
7 for k = 1:numel(femTests)
8     feval(femTests{k});
9 end
```

12.2 What each test proves

Test	Pass criterion	Interpretation
test_module2_zero_charge_2d	$\max \phi < 10^{-12} \text{ V}$	Zero source and zero contact potential produce the zero solution.
test_module2_linear_potential_2d	Maximum nodal error below 10^{-10} V	Linear T3 basis reproduces the exact charge-free linear solution.
test_module2_uniform_space_charge_2d	Relative maximum potential error below 5×10^{-3}	Source assembly approaches the analytical one-dimensional quadratic solution on the default mesh.
test_module2_localized_defect_charge_2d	Finite potential, positive maximum source, and positive maximum potential	Localized positive charge produces a physically plausible response. This is a sanity test, not a convergence proof.

Every test calls `setup_project_paths`, disables plots and MAT-file saving, calls the numerical solver directly, and uses MATLAB `assert` statements. A printed `passed` line means all assertions in that function completed.

12.3 Batch-test command

```
matlab -batch "cd('C:/path/to/RadiationEffects_proj2/matlab'); setup_project_paths;
test_module2_zero_charge_2d; test_module2_linear_potential_2d;
test_module2_uniform_space_charge_2d; test_module2_localized_defect_charge_2d;"
```

12.4 Mesh-refinement check

Passing one fixed-mesh test is necessary but not sufficient for a research calculation. For a convergence study, repeat the same physical case on successively finer meshes and compare a scalar or field quantity. The uniform-charge case has an analytical nodal reference:

```

1 meshSizes = [21, 41, 81, 161];
2 errors = zeros(size(meshSizes));
3
4 for k = 1:numel(meshSizes)
5     params = default_module2_params('uniform_space_charge');
6     params.nx = meshSizes(k);
7     params.ny = 1 + (meshSizes(k)-1)/2;
8     r = solve_poisson_defect_space_charge_2d(params);
9
10    x = r.mesh.nodes(:,1);
11    exact = (params.rho_uniform/(2*params.eps_si)) .* ...
12           x .* (params.Lx-x);
13    errors(k) = max(abs(r.phi-exact)) / max(max(abs(exact)), eps);
14 end
15
16 loglog(meshSizes, errors, 'o-');
17 xlabel('Number of x nodes'); ylabel('Relative max error'); grid on;

```

For this special problem, the exact solution is quadratic in x , whereas the T3 approximation is piecewise linear. Error should decrease with refinement. A refinement study for a coupled, highly localized source should additionally monitor peak potential, field magnitude, integrated charge, and spatial profiles.

13 Debugging and interactive code tracing

13.1 Stop at the first MATLAB error

```

1 dbstop if error
2 result = main_module2_electrostatics('localized_defect_charge');

```

MATLAB will pause at the line that raised the error. Use the Workspace, `whos`, and the editor's step controls to inspect local variables. Clear the diagnostic stop later with `dbclear if error`.

13.2 Enter the solver at its first line

```

1 dbstop in solve_poisson_defect_space_charge_2d at 1
2 result = main_module2_electrostatics('localized_defect_charge');

```

Step through in this order: `mesh`, `rho`, `K/rhs`, constrained nodes, `Kbc/rhsbc`, `phi`, then `field`. This sequence mirrors the actual numerical dependency chain.

13.3 Inspect source code and resolution

```

1 open main_module2_electrostatics
2 open solve_poisson_defect_space_charge_2d
3 open assemble_poisson_fem_2d
4 which apply_dirichlet_bc -all

```

If edits are not reflected in execution, check `which -all`, then use `clear functions` and `rehash path`. Do not continue until MATLAB resolves the intended project copy.

14 Common failure modes and diagnostic actions

Symptom	Likely cause	Action
Undefined function for a Module 2 file	Project paths were not added, or MATLAB is in the wrong extracted copy.	Run <code>setup_project_paths</code> ; inspect <code>which ... -all</code> .
Unknown Module 2 case name	The driver accepts only the four strings in the default-parameter switch.	Use an exact built-in name or call the solver with an overridden parameter structure.
Triangle has non-positive area	Connectivity was changed to clockwise order, or the mesh contains a degenerate element.	Inspect the reported element coordinates and restore counter-clockwise connectivity.
Singular or nearly singular matrix warning	No Dirichlet reference potential, disconnected mesh, or invalid constraints.	Confirm <code>fixedNodes</code> is nonempty and the mesh is connected.
Changing <code>dphidn</code> has no effect	Nonzero Neumann boundary integration is not implemented.	Keep natural flux zero or extend assembly with the correct boundary integral.

Symptom	Likely cause	Action
Changing defect parameters has no effect	<code>rho_uniform</code> exists and bypasses the general source calculation.	Remove the field or start from a non-uniform-source case.
NaN or Inf in charge	Zero/invalid Gaussian width or nonfinite input concentration.	Require positive <code>Cdef_sigma_x/y</code> and inspect every source field.
Unexpected potential sign	Charge sign, effective defect charge, or field convention was misread.	Inspect <code>rho</code> ; remember positive source with grounded contacts gives a positive potential, while $\mathbf{E} = -\nabla\phi$.
Unexpected corner potential	Two Dirichlet sides prescribe different values at one corner.	Remove the conflict or account for left/right/bottom/top first-occurrence precedence.
No output files after a direct solver call	Saving, plotting, and summary writing are driver responsibilities.	Call the three output functions explicitly or use the top-level driver.
Plot creation fails in a headless environment	Graphics renderer is unavailable even for invisible figures.	Use the direct solver route and omit <code>plot_module2_result_2d</code> ; inspect numerical arrays and summaries.
Excessive memory or long solve time	<code>nx</code> and <code>ny</code> create too many nodes/elements for the environment.	Start with the defaults, refine gradually, and monitor <code>whos result</code> .
Unphysical magnitude	A concentration or length was entered in centimeters, micrometers, or cubic centimeters without SI conversion.	Convert lengths to meters and concentrations to m^{-3} before solving.

15 Adding a new standalone case safely

There are two useful levels of customization.

15.1 One-off experiment

Do not edit the parameter factory. Create and override a structure in a script, call the solver, inspect the result, and save it under a unique name. This is the recommended exploratory path because it cannot silently alter the four regression cases.

15.2 New named regression or demonstration case

For a permanent named case:

1. Add a new `case` branch in `default_module2_params.m`.
2. Set every field that differs from the common defaults.
3. Optionally add a convenience wrapper in `matlab/cases/`.
4. Add a test in `matlab/tests/` with a quantitative pass criterion.
5. Run all four existing FEM tests to detect regressions.
6. Run the new case through the top-level driver and inspect every output.
7. Document the case name, units, reference solution or expected trend, and limitations.

Adding only a wrapper in `matlab/cases/` does not make the name acceptable to the current top-level driver; the switch in `default_module2_params.m` remains authoritative.

16 Preparing for Module 3 coupling

The standalone source builder currently constructs

$$\rho_{\text{def}}(x, y) = q z_{\text{def}} C_{\text{def}}(x, y) \quad (16.1)$$

from an internal Gaussian. In the coupled architecture, Module 3 should instead supply one or more nodal concentration fields $C_i(x, y, t)$ and Module 2 should evaluate

$$\rho_{\text{def}}(x, y, t) = q \sum_i z_i C_i(x, y, t). \quad (16.2)$$

Before implementing that interface, define and test:

- whether Modules 2 and 3 share exactly the same node ordering and mesh;
- the interpolation rule when their meshes differ;

- concentration units and effective charge signs;
- species ordering and metadata;
- conservation or interpolation error checks;
- behavior at times when no defect field is available;
- whether carrier and dopant contributions are supplied independently or already included in an imported total-charge map.

Do not overwrite `Cdef_peak` with an array and assume the existing Gaussian expression will accept it correctly. Add an explicit nodal-field branch with dimension and unit validation, then protect it with a charge-consistency test.

17 Recommended verification ladder before using a result

Use the following order when changing the FEM code:

1. **Path check:** verify that MATLAB resolves the intended files.
2. **Zero solution:** run `test_module2_zero_charge_2d`.
3. **Boundary and sign check:** run `test_module2_linear_potential_2d`.
4. **Source integration check:** run `test_module2_uniform_space_charge_2d`.
5. **Localized-source sanity:** run `test_module2_localized_defect_charge_2d`.
6. **Algebraic checks:** inspect residual, symmetry, fixed-node error, and finiteness.
7. **Mesh refinement:** establish convergence of the reported quantity.
8. **Physical scale check:** compare charge, potential, and field magnitudes with an independent estimate.
9. **Coupling check:** when importing fields, verify units, interpolation, sign, and total-charge consistency.

Interpretation rule. A successful sparse solve demonstrates that MATLAB found a vector satisfying the assembled algebraic equations. Analytical tests, convergence studies, dimensional checks, and coupling consistency are what establish that those equations represent the intended physical problem.

18 Minimal handoff checklist

Before sending or presenting a Module 2 FEM result, record:

- repository/archive version and MATLAB version;
- case name and any parameter overrides;
- L_x , L_y , `nx`, and `ny`;
- permittivity and every nonzero charge contribution;
- boundary types and values;
- passed tests and mesh-refinement evidence;
- algebraic residual and fixed-boundary error;
- output filenames and the exact quantity plotted;
- known limitations, especially prescribed sources and absent nonzero-Neumann support.

19 Next implementation note

The next document in this documentation sequence should be a separate **Module 2 PINN implementation note**. It should cover the PINN driver, the six functions in `matlab/pinn/`, Deep Learning Toolbox requirements, collocation and boundary datasets, loss weights, optimizer options, FEM anchors, prediction, verification figures, test commands, and training diagnostics. Those topics are intentionally not mixed into this FEM note.